

GitHub AI Agent v1.3

Health Score and Budget Guardrails

Production QA and business reference | Verified July 15, 2026

Purpose and production scope

This document helps QA and business users understand two parts of the production experience:

- 1. What the Health Score means and how the application calculates it.
- 2. What users can expect when a budget or scan limit affects an analysis.

The behavior described in this guide was verified on **July 15, 2026**.

Item	Production value
Public release	v1.3
Image	github-agent:v1.3-ef43092aadf8
Git revision	ef43092aadf801cbb394a37426c5ef5ddd6226da
Review method	Legacy production workflow

Scope note

- Behavior from the Local Evidence Review Harness v2 is not included because it is not active in production.

Quick answer

Key points

- The Health Score ranges from 0 to 100 and is based on findings in the files reviewed during one run. The AI identifies findings, but fixed application rules calculate the score.
- A higher score means the reviewed files contained fewer findings or findings of lower severity. It does not prove that the repository is secure or defect-free.
- A **budget-blocked run** does not start and produces no score, report, or pull request (PR).
- A **limited scan** reviews only part of the repository. It can produce a report, but it cannot create an automated PR.

1. Understanding the Health Score

What each score means

Score shown	Label	Plain-language meaning	Recommended user action
85-100	Excellent	The reviewed files contain few findings or only low-impact findings.	Review the coverage and remaining findings before deciding that no work is needed.
70-84	Good	The reviewed files are generally healthy, but some findings require attention.	Review High and Medium findings and decide which should be addressed first.
50-69	Needs Work	The analysis found enough risk or quality concerns to reduce confidence.	Review the most serious findings before relying on automated fixes or merging a PR.
0-49	Critical	The reviewed files contain serious findings or a high number of findings.	Start with the highest-severity findings and confirm them manually.

Important interpretation notes

- If at least one Critical finding exists and the score is 50 or higher, the main card displays **Needs Work**. This prevents a relatively high numeric score from masking the presence of a Critical finding.
- A **Critical** score label does not always mean the report contains a finding with Critical severity. It can also result from High findings combined with additional lower-severity findings.
- Some smaller charts change color at 80 and 60. Their color can differ from the main label, but the numeric score remains unchanged.
- A limited scan reports a score only for the files reviewed. Users should not interpret it as a score for the entire repository.
- If no source file can be reviewed, the UI should mark the result as incomplete rather than present a reliable Health Score.

How the score is calculated in plain language

The score starts at 100. Findings remove points according to severity. Starting with the fifth finding, the total number of findings also removes a small number of extra points. A missing or unrecognized severity is treated as Medium.

A High finding is treated as Medium when it relates only to maintainability, such as formatting, naming, documentation, dead code, unused code, readability, or unnecessary casts. It remains High when it also indicates a security, authorization, credential, injection, data-loss, crash, or similar runtime risk. This prevents style-only findings from reducing the score as heavily as security or runtime risks.

Severity used for scoring	Points removed per finding
Critical	24
High	9
Medium	4
Low	1

Exact scoring calculation

Calculation flow

- **1. Normalize severity:** Treat an unrecognized severity as Medium. Treat a maintainability-only High finding as Medium.
- **2. Calculate the severity penalty:** Multiply each finding count by its severity weight.
- **3. Calculate the volume penalty:** After the first four findings, remove one additional point for every two findings.
- **4. Apply the severity floor:** Lower-risk findings cannot push the finding-based score below the documented minimum.
- **5. Blend quality-area scores:** Use 70% of the finding-based score and 30% of the quality-area average. The blend may lower the score, but it can never raise it.

```
severity_penalty =
  (Critical x 24) + (High x 9) + (Medium x 4) + (Low x 1)

volume_penalty = max(0, floor((total_findings - 4) / 2))

raw_finding_score = 100 - severity_penalty - volume_penalty
finding_based_score = max(raw_finding_score, applicable_severity_floor)
```

Key rule

- **The final score can never be higher than the finding-based score.** Quality-area scores can reduce the result, but they cannot offset identified findings.

The volume penalty removes one additional point for every two findings after the first four. The result is rounded to a whole number and constrained to the range from 0 to 100.

The following floors prevent many lower-risk findings from producing a misleading score of zero:

Most serious finding present	Lowest possible finding-based score
Critical	0
High, with no Critical findings	35
Medium, with no High or Critical findings	55
Low only	70

The application uses the raw calculated score or the applicable floor, whichever is higher.

The code quality, security, and performance steps can each return a score. The application averages only valid positive scores; missing, invalid, and zero values are ignored.

When findings exist, the application applies the following blend:

```
adjusted_score = round((finding_based_score x 0.70) + (quality_area_average x 0.30))
final_score = min(finding_based_score, adjusted_score)
```

When no findings exist, the quality-area average becomes the base score. A clean analysis can then receive up to five additional points, one per generated test, without exceeding 100.

Worked score examples

Example 1 - One Medium finding

- **Inputs:** 1 Medium finding; quality-area average = 90.
- **Severity penalty:** 1 x 4 = 4. **Volume penalty:** 0, because there are fewer than five findings.
- **Finding-based score:** 100 - 4 - 0 = 96.
- **Adjusted score:** round((96 x 0.70) + (90 x 0.30)) = round(94.2) = 94.
- **Final score:** min(96, 94) = **94, Excellent.**

Example 2 - Severity and volume penalties together

- **Inputs:** 1 High, 3 Medium, and 2 Low findings; 6 findings in total; quality-area average = 60.
- **Severity penalty:** (1 x 9) + (3 x 4) + (2 x 1) = 23.
- **Volume penalty:** floor((6 - 4) / 2) = 1.
- **Finding-based score:** 100 - 23 - 1 = 76. The High-severity floor of 35 does not change it.
- **Adjusted score:** round((76 x 0.70) + (60 x 0.30)) = round(71.2) = 71.
- **Final score:** min(76, 71) = **71, Good.**

2. Choosing an analysis mode

The selected mode changes the expected cost, file coverage, and test-generation limits. It does not change the scoring formula.

Mode shown in the UI	Best used for	Code files reviewed	Test limit	What the user should expect
Fast	A quick first look at a small repository	20, or 30 when the repository has over 400 files	15 files, up to 5 modules or 3 minutes	Lowest cost estimate and fastest result. More likely to become a limited scan when the repository is larger than the file limit.
Thorough	A balanced review for a normal-sized repository	50, or 60 when the repository has over 400 files	40 files, up to 10 modules or 6 minutes	Wider coverage than Fast with a standard cost multiplier. This is the balanced option for most reviews.
Deep	The widest available review for a larger or more complex repository	80, or 90 when the repository has over 400 files	70 files, up to 15 modules or 10 minutes	Broadest coverage and longest test budget. Its higher cost estimate makes a pre-run budget block more likely.

For repositories with more than 400 files, each test-file limit increases by 10, up to a maximum of 90. No mode guarantees that every file will be reviewed. Code analysis has an absolute maximum of 100 files. The report's coverage count is the source of truth.

Examples

- **Fast with 12 readable and analyzable files:** The run can produce a full scan because all files fit within the limit.
- **Fast with 80 readable and analyzable files:** The run reviews a selected set of files and should appear as a limited scan.
- **Deep with the same 80 files:** The run can review all files when they are accessible and within the size limit.
- **Deep with a high cost estimate:** The run can still be blocked because the selected mode does not override the USD 1.00 per-run limit.

3. Understanding budget and scan limits

Cost check before analysis starts

Limit	Production value
Estimated cost per analysis	USD 1.00
Monthly recorded usage per user	USD 10.00
Enforcement	Hard block

Monthly usage includes recorded analysis and chat costs for the user during the current UTC calendar month.

The initial estimate uses repository size, selected mode, analysis type, provider, and model price:

```
base tokens = max(2,200, repository size in KB x 8)

mode shown in the UI: Fast x 0.60 | Thorough x 1.00 | Deep x 1.35
analysis type: comprehensive x 1.20 | security x 1.10 | custom x 1.15 | other x 0.95
```

The application blocks the run when:

- The estimated run cost is more than USD 1.00.
- The user has already reached USD 10.00 for the month.
- The new estimate would take the user's monthly total above USD 10.00.

Expected budget-block message

- The UI shows **This analysis run exceeds the configured budget guardrails.**
- No analysis begins, and no score, report, AI request, or PR is created.

The amount shown before a run is an estimate. Final cost uses actual recorded model usage, so it can differ from the estimate.

File selection during an allowed run

The production workflow first retrieves content for up to 20 likely relevant files, with a maximum of 100,000 bytes per file. It can retain basic information for up to 1,000 files in Fast mode, 2,000 in Thorough mode, or 4,000 in Deep mode. Basic information does not mean the code was fully reviewed.

Code analysis can retrieve additional selected files, up to 50,000 bytes each. It favors likely entry points, configuration files, authentication and security files, API routes, components, pages, and deployment files. It then selects other high-priority and representative files until the chosen mode's limit is reached.

When the application can read more files than the selected mode can review, when file information is shortened, or when sampling is used, the run becomes a **Limited scan**. The report can complete, but automated PR creation is disabled because the full available scope was not reviewed.

Limited-scan UI requirements

- Show a **Limited scan** label.
- Show coverage, such as **Analyzed 54 of 248**.
- Clarify that the score applies only to the reviewed scope.
- Explain why an automated PR is unavailable.
- Offer a deeper mode when the cost policy allows it.

Fix limits after a full scan

On a full scan, automated fixing attempts to address at most six findings across three files. It handles no more than four findings per file and spends no more than 45 seconds per file. Files longer than 20,000 characters and files detected as binary are left for manual review.

These limits do not mean the analysis failed. A full scan can create a PR with a safe subset of fixes. The UI or PR notes should explain what was not included and why. A limited scan cannot create an automated PR.

4. What QA should expect

Outcome	Score and report	PR behavior	Expected message or action
Full scan	Covers all files the application can read and analyze	Allowed only when a meaningful fix exists	Show a Completed status. Show a PR action only when a PR exists.
Limited scan	Covers only the reviewed selection	Blocked	Show coverage, the limitation, the score scope, and a deeper-scan option.
Pre-run budget block	Not produced	Blocked	Explain the cost limit and suggest reducing the mode, scope, or configured budget.
Test limit reached	The main report can still complete	Not blocked by this limit alone	Show generated and planned test counts.
Fix limit reached	The main report remains complete	May contain a safe subset	Show attempted and deferred findings with reasons.
No source can be reviewed	Withheld or marked unreliable	Blocked	Explain whether access, file type, empty content, size, or encoding caused the problem.

Final interpretation

- The Health Score describes the evidence reviewed in one run. It is not a certification that the repository is defect-free or secure.